Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Senkronizasyon Mekanizmalarının Değerlendirilmesi
  - Bir senkronizasyon mekanizması ortaya atıldığında, işlerliğini ölçmek ve test etmek gerekir.
  - Temel özellikler: Mutual exclusion sağlamalı, Bounded waiting (sınırlı bekleme) sağlamalı, gereksiz beklemenin önüne geçmeli.
- Klasik Senkronizasyon Problemleri
  - Bu problemler, bir senkronizasyon mekanizmasının uygunluğunu ölçen test problemleridir.
  - Gerçek hayatta da uygulama gerçekleştirirken karşılaşılan senaryolara denk düşer.
- Bounded-Buffer (Sınırlı Tampon) Problemi
  - Adı üzerinde sınırları belirli bir buffer var. Her slot bir item (karakter, integer, struct, object) tutar.
  - Toplam N item tutulabilir.
  - **Çözüm**: 3 semafor kullanılır.
    - `mutex` (1): Karşılıklı dışlama. Binary semaphore (1 ile başlatılır).
    - `full` (0): Dolu slot sayısı. 0 ile başlatılır.
    - `empty` (N): Boş slot sayısı. N ile başlatılır.
  - **Producer**:
    ```
    while (true) {
      wait(empty);     // boş yer var mı?
      wait(mutex);     // kritik bölgeye gir
      // üret, buffer'a ekle
      signal(mutex);   // kritik bölgeden çık
      signal(full);    // dolu sayısını arttır
    }
    ```
  - **Consumer**:
    ```
    while (true) {
      wait(full);      // dolu item var mı?
      wait(mutex);     // kritik bölgeye gir
      // tüket, buffer'dan al
      signal(mutex);   // kritik bölgeden çık
      signal(empty);   // boş sayısını arttır
    }
    ```
  - **Önemli özellik**: `full + empty = N` her zaman. Çünkü dolu + boş = toplam.
  - Bu yapı, istediğiniz sayıda consumer process çalıştırmanıza izin verir; hepsi aynı kod parçasını çalıştırır.
- Readers-Writers (Okuyucular-Yazıcılar) Problemi
  - Veritabanı modeli: SELECT (okuma), INSERT/UPDATE/DELETE (yazma).
  - Reader: Sadece okur, değişiklik yapmaz.
  - Writer: Hem okur hem yazabilir.
  - **Problem**: Okuma isteği varsa bekletilmemeli (okuma veri değiştirmez). Ancak yazma devam ediyorsa okuma beklemeli.
  - **Çözüm**: 2 semafor + 1 integer değişken.
    - `rw_mutex`: Yazıcıların kendi aralarındaki karşılıklı dışlama.
    - `mutex`: `readcount`'u korumak için.

    - `readcount` (0): Aktif okuyucu sayısı.
  - **Reader**:
```
while (true) {
  wait(mutex);
  readcount++;
  if (readcount == 1) wait(rw_mutex);  // ilk okuyucu yazıcıyı beklet
  signal(mutex);
  // oku
  wait(mutex);
  readcount--;
  if (readcount == 0) signal(rw_mutex);  // son okuyucu yazıcıyı serbest bırak
  signal(mutex);
}
```
  - **Writer**:
```
while (true) {
  wait(rw_mutex);
  // yaz
  signal(rw_mutex);
}
```
  - **Varyasyonlar**: Reader-priority (yazıcılar starvation'a uğrayabilir) veya Writer-priority (okuyucular starvation'a uğrayabilir).
  - Bazı OS'lerde reader-writer lock'ları doğrudan implement edilmiştir (POSIX, Windows).
- Dining-Philosophers (Yemek Yiyen Filozoflar) Problemi
  - 5 filozof yuvarlak masada. Her filozofun sağında ve solunda birer chopstick (çubuk) var.
  - Hayat döngüsü: Düşün → Acık → Yer → Düşün. Düşünme = ready, yeme = running, açlık = waiting.
  - Yemek yemek için 2 çubuk (sol + sağ) gerekir.
  - **Naif Çözüm (Deadlock!)**: Her filozof önce sol, sonra sağ çubuğu alır. Tüm filozoflar aynı anda sol çubuğu alırsa, hiçbirinin sağ çubuğu kalmaz → deadlock.
  - **Asimetrik Çözüm**: Tek numaralılar önce sol, çift numaralılar önce sağ. Ancak 0,2,4 = 3 kişi, 1,3 = 2 kişi; dengesizlik. Kaynaklar eşit paylaşılmıyor.
  - **İyileştirilmiş Asimetrik Çözüm**: Her turda 0,2,4 sağdan, 1,3 soldan alır; sonraki turda tersi. Bu sayede iki tam turda eşit pirinç dağıtılır.
  - **Kaynak Sıralama (Resource Hierarchy)**: Çubukları numaralandır. Her filozof artan sırada istesin. Bu circular wait'i engeller.
- Monitörler
  - Semaforlardan daha üst seviye, daha soyut bir senkronizasyon yapısı.
  - Programcıya yakın; kullanım hatalarını azaltır.
  - **Özellikler**:
    - High level abstraction: Kolay ve etkili senkronizasyon.
    - Sadece bir proses aynı anda aktif olabilir (otomatik mutual exclusion).
    - Kütüphane olarak hazır kullanılabilir.
  - **Yapı**:
    - Shared variables (paylaşılan değişkenler).
    - Procedures (prosedürler, metotlar).
    - Initialization code (constructor).
  - **Condition Variables**: `wait` ve `signal` operasyonları. Semaforlardan farklıdır (değer sayılmaz).
    - `x.wait()`: Proses bloklanır.
    - `x.signal()`: Bekleyen prosesi uyandırır (yoksa etkisiz).
  - **Avantaj**: Programcının yanlış kullanma riski azalır.
  - **Dezavantaj**: Her senkronizasyon problemi için yazılan monitör kodu yeterince etkili olmayabilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Klasik Problemlerin Test Amaçlı Kullanımı
  - Hoca vurgular: "Bu problemler aslında ortaya atılan bir sinkronizasyon mekanizma-

sının sinkronizasyon için uygunluğunu ölçen problemler. Üç tane temel problemimiz var. Bunlar bilişim dünyasında da uygulama gerçekleştirirken, uygulamaların birbirleriyle iletişimi veya uygulama içerisindeki alt parçaların, uygulamaya ayrılmış kaynakları kullanımı ile alakalı senaryolarda da karşımıza çıkıyor."

- Mutex Semaphore'unun Önemi
  - "Mutex adında bir semaforum var. Wait ile bu Mutex üzerinde bekleme yapıyoruz. Eğer semaforun değeri sıfırdan büyükse ben kritik bölgeye gireceğim ve işlemimi yapacağım."
- Reader-Writer'da readcount'un Kritik Rolü
  - "Bütün prosesler, okuma yazmak isteyen bütün prosesler readcount'u bir arttıracaklar. İşlemleri bittiği zaman da bir eksilecekler. Read count'u bir arttırıyorum. Eğer read count'u bir ise, yani okuma yapacak ilk proses o ise, o zaman bu prosesin neyi beklemesi lazım? Writer'ları."
- Dining Philosophers'ta Deadlock
  - Hoca vurgular: "Bu yapı eğer dikkatli bir şekilde gerçekleştirilmez ise büyük problemlere yol açar. Şimdi bir örnek vermiştim. Hani buradaki şu anda anlattığımız çözüm üzerine olan çözüm görülecek. Ve bakın önce konuştuğumuz şey, önce chopstick i'yi alıyor. What is the problem with this algorithm? Nedir? Otomatikman deadlock."
- Semaforların Doğru Kullanımı
  - Hoca vurgular: "Semaforları çok güzel, çok iyi. Ancak semaforları kullanırken, kullanım kurallarına dikkat etmez ise otomatikman problemlere sebep oluyoruz."
- Monitörün Yapısal Özelliği
  - "Monitorda herhangi bir anda sadece bir proses aktif olabilir dedik ya, o zaman bakın monitorun girişinde bir kuyruk var. Sadece bir tanesi aktif olduğuna göre, ben de bu kuyruğu sıralı bir kuyruk olarak modelleyebilirim."

Kısa Tekrar Notları

- Bounded buffer: mutex (1) + full (0) + empty (N). full + empty = N.
- Readers-writers: rw_mutex + mutex + readcount. İlk okuyucu rw_mutex'i bekler, son okuyucu serbest bırakır.
- Dining philosophers: 5 filozof, 5 çubuk. Asimetrik çözüm veya kaynak sıralama.
- Monitör: üst seviye senkronizasyon yapısı, sadece bir proses aktif.
- Condition variable: wait (bloklan), signal (uyandır).

Detaylı Açıklamalar

Ders 11, senkronizasyon konularının devamında klasik senkronizasyon problemlerini ve monitör yapılarını ele alır. Bu ders, geçen haftaki semafor ve mutex konularının pratik uygulamalarını gösterir.

Senkronizasyon mekanizmalarının işlerliğini test etmek için üç klasik problem kullanılır:

Bounded-Buffer (Producer-Consumer) Problemi: Bir üretici (producer) ve bir tüketici (consumer) arasında sınırlı bir buffer üzerinden veri alışverişi yapılır. Üç semafor kullanılır: `mutex` (binary, 1 ile başlar), `full` (counting, 0 ile başlar, dolu slot sayısı), `empty` (counting, N ile başlar, boş slot sayısı). Producer önce `empty`'yi bekler (boş yer var mı), sonra `mutex`'a girer, üretir, çıkar, sonra `full`'ı arttırır. Consumer ise `full`'ı bekler (dolu item var mı), sonra `mutex`'a girer, tüketir, çıkar, sonra `empty`'yi arttırır. Önemli özellik: `full + empty = N` her zaman.

Readers-Writers Problemi: Birden fazla okuyucu aynı anda veri okuyabilir, ancak yazıcı tek başına çalışmalıdır (okuyucu veya başka yazıcı olamaz). Reader, veri değiştirmez; writer hem okur hem yazar. Çözümde `readcount` (aktif okuyucu sayısı) değişkeni, `rw_mutex` (yazıcılar için) ve `mutex` (readcount'u korumak için) semaforları kullanılır. İlk okuyucu `rw_mutex`'i bekler; son okuyucu onu serbest bırakır.

Dining-Philosophers Problemi: 5 filozof yuvarlak masada yemek yer. Her filozofun sağında ve solunda birer chopstick (çubuk) var. Yemek yemek için 2 çubuk gerekir. Naif çözüm (her filozof önce sol, sonra sağ) deadlock'a yol açar. Asimetrik çözümler veya kaynak sıralama ile çözülür. Bu problem, deadlock'un klasik örneğidir ve dört deadlock koşulunun (mutual exclusion, hold and wait, no preemption, circular wait) hepsinin birden sağlandığını gösterir.

Monitörler, semaforlardan daha üst seviye bir senkronizasyon yapısıdır. Semaforlarla doğru kullanım sorunları vardır (sıra karıştırma, unutma). Monitör, sınıf (class) benzeri bir yapıdır: paylaşılan değişkenler + bu değişkenler üzerinde çalışan prosedürler içerir. Monitörde herhangi bir anda sadece bir proses aktif olabilir; bu mutual exclusion'ı otomatik sağlar. Koşul değişkenleri (condition variables) ile senkronizasyon sağlanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.